UNIVERSITY OF GHANA

COLLEGE OF BASIC AND APPLIED SCIENCES

SCHOOL OF ENGINEERING SCIENCES

DEPARTMENT OF COMPUTER ENGINEERING

SECOND SEMESTER 2025/2026 ACADEMIC YEAR

COURSE CODE AND TITLE: CPEN 438 – ADVANCED COMPUTER ORGANIZATION AND ARCHITECTURE

PROJECT TITLE: WIRING THE DATA CENTRE: INTERCONNECTION NETWORKS FOR A SIMULATED ACCRA WAREHOUSE-SCALE CLUSTER

WEEK 4 FINAL TECHNICAL REPORT

GROUP 1

GROUP MEMBERS:

1. **AYERTEY VANESSA ESINAM - 11264010**
2. **PEGGY ESINAM SOMUAH - 11049523**
3. **BERNARDINE ADUSEI-OKRAH - 11123762**
4. **MUHAMMAD NURUL HAQQ ABDUL BASIT MUNAGAH - 11117536**
5. **FAROUK SEDICK - 10947554**

LECTURER: PROF. ROBERT SOWAH

DUE DATE: 2ND SEPTEMBER, 2026

Wiring the Data Centre: Interconnection Networks for a Simulated Accra Warehouse-Scale Cluster

CPEN 438 — Advanced Computer Architecture Systems and Design

Project 13, Group 1

Assigned configuration: 16 nodes · ring(16) · mesh(4×4) · seed 1301

Abstract

We build a cycle-accurate, flit-level interconnection-network simulator and use it to measure how topology choice governs latency, throughput and saturation in a simulated 16-node warehouse-scale cluster. Three topologies are compared under two traffic patterns: a 16-node ring, a 4×4 mesh, and — as the Level-3 Advanced extension — a 4×4 folded torus. Hand-computed diameter and bisection bandwidth are verified against the simulator's own topology construction before any traffic experiment is trusted, and every packet is accounted for by an explicit ledger that reconciles exactly across 391 sweep runs with zero violations.

Three results stand out. First, under uniform-random traffic the measured saturation points track the structural metrics closely: the ring saturates at 0.28 flits/node/cycle, the mesh at 0.70, and the torus at 0.94, in the same order as their bisection bandwidths of 2, 4 and 8 links. Second, under hot-node-skewed traffic that ordering collapses — the ring and mesh both saturate at 0.20 — because the binding constraint moves from the bisection cut to the single unit-bandwidth channel feeding each hot node, a constraint every topology shares. The classical bisection bound overestimates the achievable rate by up to 6.4× in this regime, while an explicit channel-load model predicts it to within 5–33%. Third, the torus's structural advantage is not free: because a torus needs a dateline rule for deadlock freedom while a mesh does not, an equal *total* virtual-channel budget leaves the torus with half the usable buffering and it *loses* to the mesh (0.60 against 0.70). Given equal *usable* channels it wins decisively — 0.94 against 0.70, a 34% improvement — at a cost of 2.00× wire length and 1.87× network area.

Our innovation component, end-to-end traffic-class isolation, cuts background-traffic latency under hot-node load by 71.4× on the mesh and 59.6× on the ring, against 1.5× and 4.9× for a buffers-only control — establishing that the gain comes from isolation rather than from added buffering.

I. Introduction

The network that connects the nodes of a multiprocessor or a data-centre rack is one of the most consequential and least intuitive architectural decisions in the system. A topology's diameter sets the floor on communication latency; its bisection bandwidth sets the ceiling on sustained throughput; and its node degree and wire length set the cost in area and power. These three pull against each other, and the right answer depends on the traffic the machine

actually carries.

This project instantiates that trade-off concretely. We model a simplified Accra data centre serving concurrent web, API and database traffic, and we ask a specific question: *given a fixed 16-node budget, which interconnection topology sustains the most offered load before queueing delay grows without bound, and can that behaviour be predicted from the topology's structure alone?*

The simulator is the instrument, not the contribution. The contribution is the explanation — connecting hand-computed structural properties to measured saturation behaviour, and being honest where the two diverge.

A. Contributions

1. A cycle-accurate flit-level simulator with credit-based wormhole flow control, virtual channels, and an explicit packet ledger that reconciles exactly at every measured point (§III, §VIII).
2. A verified comparison of ring, mesh and folded torus under uniform-random and hot-node-skewed traffic, with hand-computed structural metrics gated against the simulator's own construction before any traffic result is reported (§V-A).
3. An analytical channel-load model that predicts saturation far more accurately than the classical bisection bound, and an account of *why* the bisection bound fails under skewed traffic (§IV, §V-D).
4. A folded-torus extension with an area and wiring-cost accounting calibrated against Dally and Towles's reported 6.6% network-area overhead, including the finding that deadlock freedom, not wiring, is what actually limits the torus at a fixed buffer budget (§VI).
5. An innovation component — end-to-end traffic-class isolation — evaluated against two deliberate controls that separate the effect of isolation from the effect of simply adding buffers (§VII).

II. Background and Related Work

A. Structural metrics

Network diameter is the maximum hop count between any two nodes; it bounds zero-load latency. **Bisection bandwidth** is the minimum number of links crossing any cut that splits the network into two equal halves; a network with low bisection bandwidth saturates early under traffic that must cross that cut, regardless of how much bandwidth exists elsewhere. Average packet latency decomposes as queueing delay plus hop count times per-hop router delay, and the **saturation throughput** is the injection rate beyond which queueing delay grows without bound.

B. Dally and Towles, "Route Packets, Not Wires" (DAC '01)

Dally and Towles argue that a structured, packet-switched on-chip network is preferable to ad-hoc global wiring on three grounds: well-controlled electrical parameters, modularity, and a better area/bandwidth trade-off — at an estimated network-area overhead of about 6.6% for a 16-tile chip using a 2D folded torus.

Our topology comparison directly instantiates this claim, and because the paper's example configuration is a 16-tile folded torus — precisely the extension we build — their 6.6% figure serves as the calibration anchor for our own area accounting (§VI-D) rather than as a loose citation. Their observation that folding equalises wire length is reproduced quantitatively in §VI-D.

C. Esmaeilzadeh et al., "Dark Silicon and the End of Multicore Scaling" (ISCA '11)

This work establishes that power and area budgets, not core count, bind future scaling. It frames our critical evaluation: as the area budget tightens, a topology that buys throughput with wire length and buffering becomes progressively harder to justify. Our finding that the torus costs 1.87× the mesh's network area for a 34% saturation improvement (§VI-D) is exactly the kind of trade-off that shifts under an area-constrained budget, and we return to it in §X.

III. System Design and Methodology

A. Flit-level packet model

Each packet is decomposed into `PACKET_FLITS = 4` flits — one head, two body, one tail. Links carry one flit per cycle per direction. Each node holds one input-buffered virtual-channel router plus a processing element with a source queue.

Flow control is credit-based wormhole: a flit advances only if the downstream virtual-channel buffer has a free slot. Virtual-channel allocation is performed by the head flit and held until the tail flit departs, so a packet is never interleaved with another on the same channel. Switch allocation grants one flit per output port per cycle, arbitrated round-robin among requesting input virtual channels.

Each cycle is evaluated as a strict two-phase read/commit sequence — credit snapshot, route computation and VC allocation, switch allocation, commit, then injection — so that no flit can traverse two routers within a single cycle. This detail matters: a naïve single-phase update silently inflates throughput.

B. Topologies and routing

All three topologies share one adjacency-list representation, so no component above the topology layer branches on topology type.

- **Ring(16):** shortest-direction routing, clockwise on ties.
- **Mesh(4×4):** deterministic dimension-order (X-then-Y) routing.
- **Torus(4×4):** dimension-order routing taking the shorter way round each

dimension, with ties broken statically toward the increasing direction.

All three routing functions are **strictly deterministic and single-path**. No routing decision anywhere in the system consults congestion. This is deliberate: an essential-tier implementation that opportunistically avoids a congested

link would blur the static/adaptive distinction that the Level-3 research challenge exists to explore, and would invalidate the comparison.

C. Traffic generation

Two patterns are required, both driven by the team seed 1301:

- **Uniform-random:** every node equally likely to be the destination, self excluded.
- **Hot-node-skewed:** a small set of "database" nodes receives a disproportionate

share, modelling real data-centre access skew. The hot set is *derived from the seed* rather than hard-coded — `hot[0] = seed mod N`, `hot[1] = (seed div N) mod N` — giving nodes {5, 1} for Group 1, with 50% of all traffic aimed at them.

The generator uses an explicit xorshift64* engine seeded through SplitMix64 rather than `rand()`, so the stream is identical on every machine and libc. A Python reference implementation reproduces the C stream bit-for-bit; this cross-language equality is asserted on every pipeline run (3000/3000 destinations matched for both patterns).

D. Deadlock freedom

The mesh under strict XY routing has an acyclic channel-dependency graph and is deadlock-free with a single virtual channel. **The ring and torus are not.**

For the ring, the clockwise channels alone form a cycle. We apply the standard *dateline* rule: the ring receives two VCs per port, packets start on the low VC, and crossing the link between node 15 and node 0 forces a packet onto the high VC. Because shortest-direction routing never exceeds $\lfloor N/2 \rfloor$ hops, a packet crosses the dateline at most once, breaking the cycle.

For the torus, *every* dimension contains a cycle. Dimension-order routing already forbids returning to the X dimension once Y has been entered, so the two dimensions cannot form a joint cycle and a single VC pair can serve both: the dateline bit is **reset at the X→Y transition**. A packet therefore crosses at most one dateline per dimension while holding the low channel, and two VCs suffice.

This design decision has a measurable cost, quantified in §VI-C.

E. Measurement methodology

Each run comprises a warm-up phase (3000 cycles), a measurement window (12 000 cycles), and a bounded drain (up to 30 000 cycles). Latency and throughput statistics are attributed only to packets *generated within the measurement window*, and any such packet still undelivered when the drain cap expires is reported as unretired rather than silently discarded.

Operational definition of saturation. The specification defines saturation only conceptually. We adopt one concrete criterion and apply it identically to every configuration:

A configuration is saturated at the first offered rate at which accepted throughput falls below 95% of offered load, **or** at which any packet is dropped from a source queue.

The reported saturation rate is the highest offered rate that was still stable and loss-free under this criterion.

IV. Analytical Model

Section L requires the latency-versus-injection-rate curves to come from an analytical model validated against measurement, not merely a replot of simulator output. Our model derives saturation from topology structure alone.

For each (topology, traffic) pair we compute the exact destination distribution *P(dst | src)*, walk every source–destination pair through the **actual routing function**, and accumulate the load on every unidirectional channel. Let γ be the load on the most heavily loaded channel, in units of the per-node injection rate. Then:

```
λ_channel   = 1 / γ
λ_eject     = EJECT_BW / max_d (ejection load at d)
λ*          = min(λ_channel, λ_eject)
T₀          = H · t_router + (L − 1)
T(λ)        = T₀ + (ρ / (1 − ρ)) · L/2,     ρ = λ / λ*
```

where *H* is the average hop count weighted by the destination distribution and the routing algorithm, *L* the packet length in flits, and the final term is an M/D/1 source-queue delay.

Crucially, γ is computed by explicit channel-load walking, **not** by assuming a uniform bisection cut. We also report the classical bisection bound $\lambda\_bisection = B / (N/2 \cdot (N/2)/(N-1))$ alongside it, because that is the bound the project brief's worked example uses — and the divergence between the two is one of our central results (§V-D).

The model is implemented identically in MATLAB (`bisection_latency_model.m`, the submitted analytical deliverable) and mirrored in Python so figures can be regenerated without a MATLAB licence. The torus routing used by the model was verified against the C implementation across all 256 source–destination pairs.

V. Experimental Results

A. Static metrics: hand computation versus simulator

Section G makes this a hard gate: hand-computed values must match the simulator's own topology construction before any traffic result is trusted. All values were computed by hand and recorded in the repository *before* the corresponding topology was implemented.

Table I — Structural metrics, hand-computed and simulator-verified

Topology	Diameter (hand)	Diameter (sim)	Bisection (hand)	Bisection (sim)	Links	Degree

ring(16)	8	8	2	2	16	2
mesh(4×4)	6	6	4	4	24	4
torus(4×4)	4	4	8	8	32	4

Diameter is computed in the simulator by breadth-first search from every node — an implementation genuinely independent of the closed-form hand derivation — and bisection by the closed form for each regular topology. Average hop counts agree to three decimal places: 4.267 (ring), 2.667 (mesh), 2.133 (torus).

The gate is enforced in code: `verify_topology` compares every value against the closed form for the configured size and exits non-zero on any mismatch, and `run_sweep` re-runs the same check before reporting any traffic result.

B. Uniform-random traffic

Table II — Measured results, uniform-random traffic

Topology	Diam	Bisect	Zero-load latency	Saturation	Peak throughput
ring(16)	8	2	8.57	0.28	0.273
mesh(4×4)	6	4	6.88	0.70	0.736
torus(4×4)	4	8	6.27	**0.94**	**0.906**

(Latency in cycles; rates in flits/node/cycle. VC budgets as in §VI-C.)

Under uniform traffic the measured ordering matches the structural prediction exactly. Zero-load latency tracks diameter and average hop count: the ring's 8.57 cycles against the torus's 6.27. Saturation tracks bisection bandwidth: 0.28, 0.70 and 0.94 for bisections of 2, 4 and 8 links.

The ring-versus-mesh result is the one the project brief predicts, and it holds: the mesh sustains 2.5× the ring's offered load before saturating, because cutting a ring anywhere severs only two links while the mesh's midpoint cut severs four. Figure 1 shows the saturation knees clearly separated.

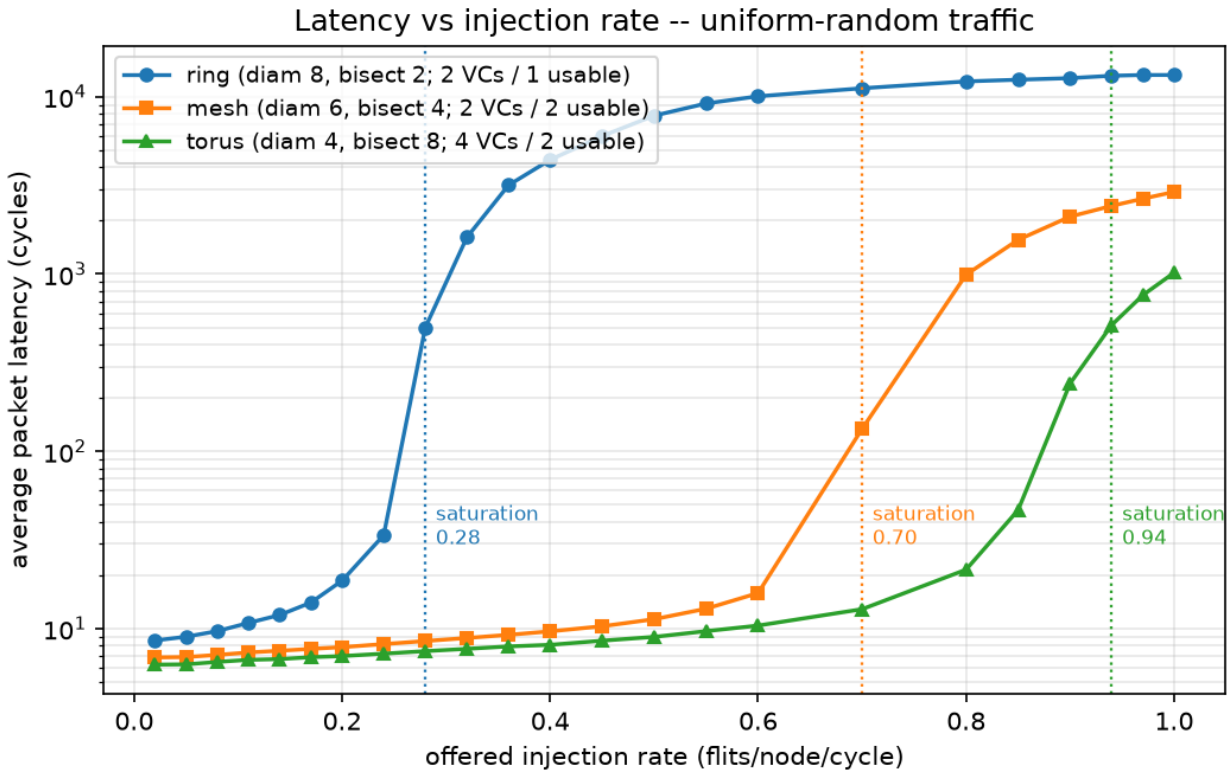

Figure 1 — Latency versus injection rate, uniform-random traffic, all three topologies. Saturation knees at 0.28, 0.70 and 0.94.

Figure 3 shows the same result as accepted throughput against offered load. Each topology tracks the ideal line until its own knee, then flattens: the ring departs first at 0.28, the mesh at 0.70, the torus last at 0.94. The vertical gap between a curve and the ideal line after the knee is offered load the network cannot absorb.

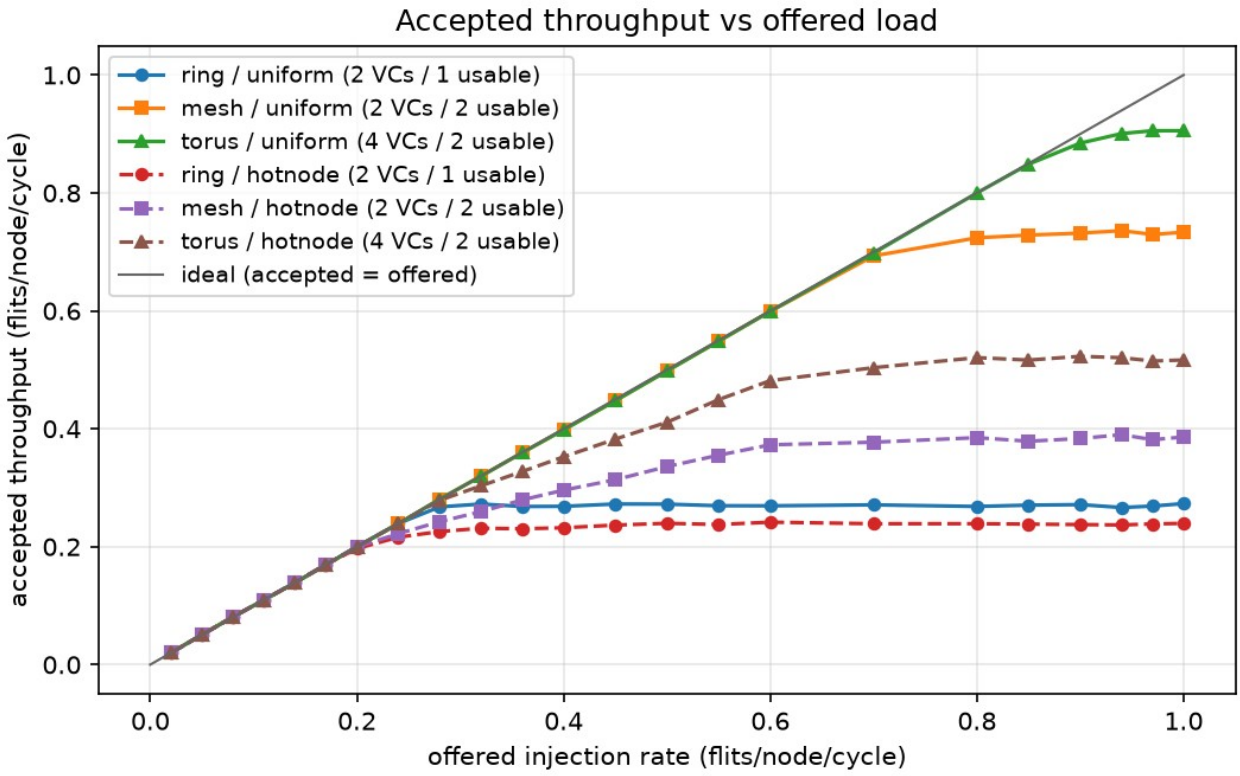

Figure 3 — Accepted throughput against offered load, both traffic patterns.

C. Hot-node-skewed traffic, and where the prediction fails

Table III — Measured results, hot-node-skewed traffic

Topology	Saturation	Peak throughput	Latency at λ=0.20
ring(16)	0.20	0.242	174.73
mesh(4×4)	0.20	0.390	23.09
torus(4×4)	0.28	0.523	8.46

The project brief predicts that the mesh-versus-ring gap should **widen** under hot-node traffic, on the reasoning that skewed traffic concentrates load precisely where bisection-limited topologies are weakest. **Our measurements contradict this.** The gap does not widen; on the saturation-rate metric it closes entirely — ring and mesh both saturate at 0.20 — and on peak throughput it narrows from 2.69× under uniform traffic to 1.61×.

This is not an implementation defect, and the channel-load model explains it precisely. Under 50% hot traffic the binding constraint stops being the bisection cut and becomes **the single unit-bandwidth channel feeding each hot node**. Under XY routing, every packet destined for node 1 (row 0, column 1) arriving from rows 1–3 must traverse the one link from node 5 into node 1. That channel is a unit-bandwidth resource that the ring possesses too, and adding bisection bandwidth elsewhere in the network does nothing to relieve it.

The evidence is in the model's own intermediate quantities (Table IV): for the mesh under hot-node traffic the classical bisection bound gives λ = 0.938, while the actual peak channel load gives 0.221 — the measured value is 0.20. The bisection bound overestimates by 4.2×. For the torus the overestimate reaches 6.4×.

The mesh advantage does not vanish entirely: it survives in peak throughput (0.390 against the ring's 0.242) and dramatically in latency at a fixed load (23.09 cycles against 174.73). But the headline saturation metric shows no advantage at all, and a report claiming otherwise would be false.

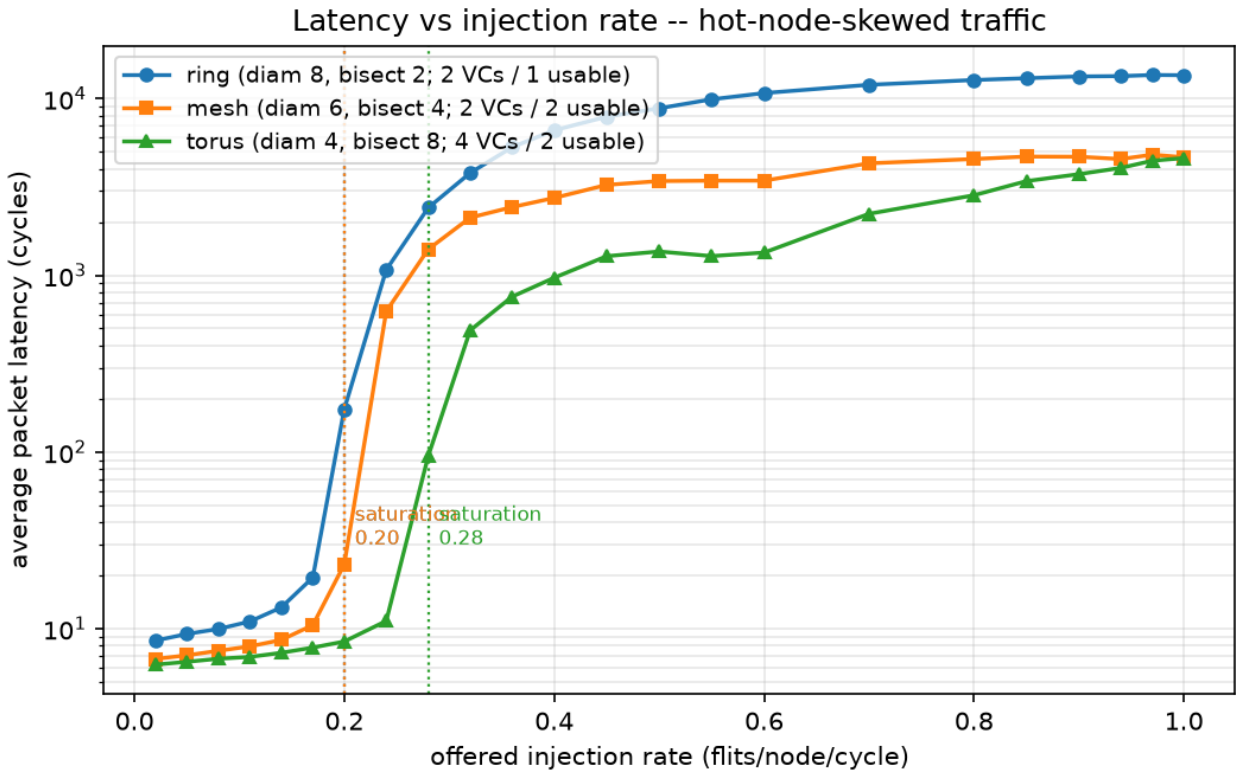

Figure 2 — Latency versus injection rate, hot-node-skewed traffic. Note the compressed spread of the saturation knees relative to Figure 1.

D. Model validation

Table IV — Analytical model versus measurement

Topology	Traffic	Avg hops	Peak chan. load	λ_channel	λ_bisection	λ*	Measured	Efficiency
ring	uniform	4.267	2.400	0.417	0.469	0.417	0.28	0.67
ring	hotnode	4.267	4.200	0.238	0.469	0.238	0.20	0.84
mesh	uniform	2.667	1.067	0.938	0.938	0.938	0.70	0.75
mesh	hotnode	2.533	4.533	0.221	0.938	0.221	0.20	0.91

torus	uniform	2.133	0.800	1.250	1.875	1.250	0.94	0.75
torus	hotnode	2.133	3.400	0.294	1.875	0.294	0.28	0.95

Two observations follow.

First, **the classical bisection bound is accurate only when the traffic actually stresses the bisection.** For the mesh under uniform traffic, λ_channel and λ_bisection coincide exactly at 0.938 — the bisection *is* the binding constraint, and the textbook bound is right. In every other row they diverge, by up to 6.4× for the torus under hot-node traffic. Reporting saturation against a bisection bound alone would have produced a badly wrong prediction in four of six cases.

Second, **measured saturation reaches 67–95% of the channel-load bound.** The shortfall is the flow-control efficiency the model does not attempt to capture: finite buffering, head-of-line blocking, and round-robin arbitration. The efficiency is systematically higher under hot-node traffic (0.84–0.95) than under uniform (0.67–0.75), because when a single channel is the bottleneck the network approaches that limit cleanly, whereas distributed contention leaves more capacity stranded.

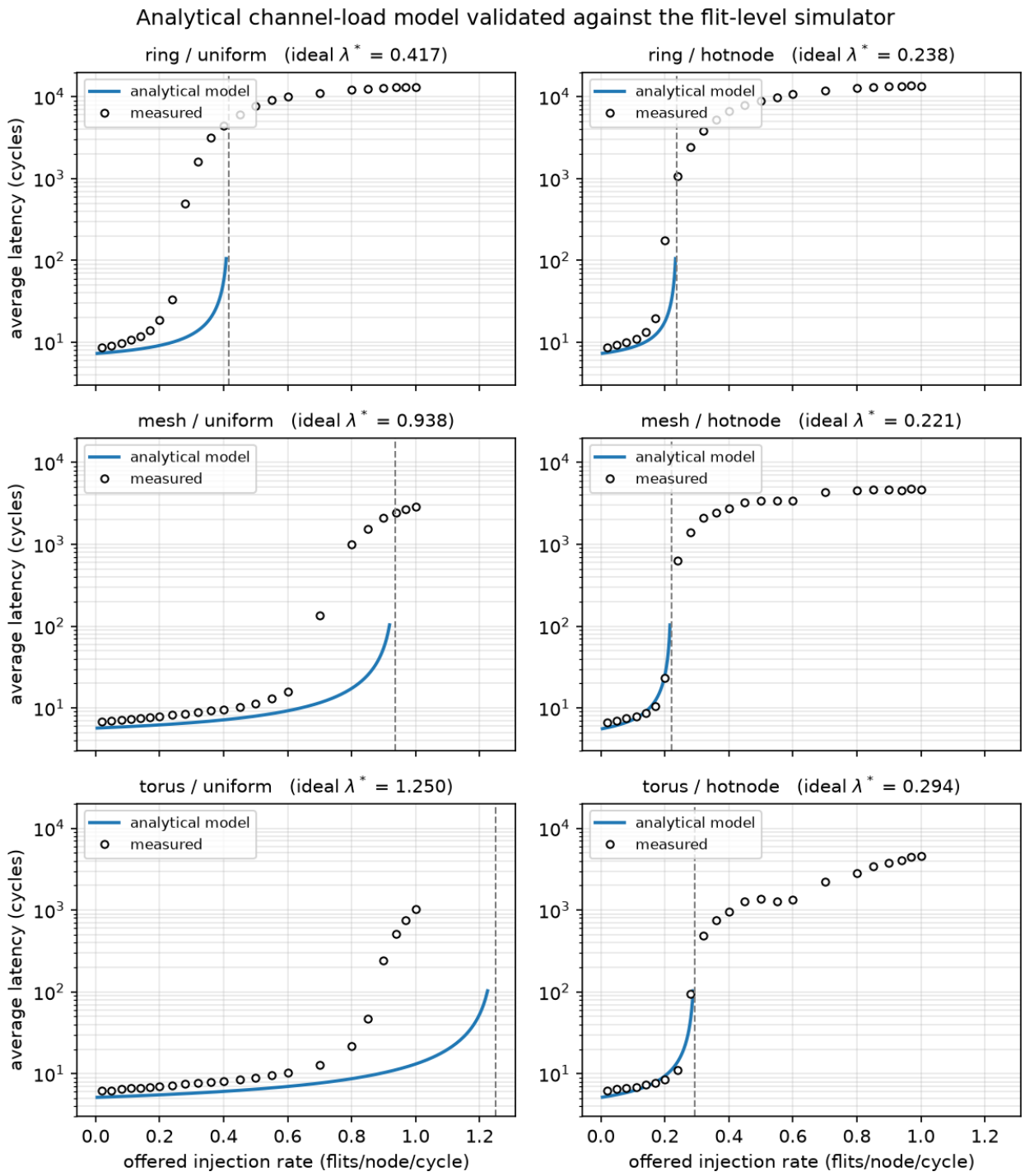

Figure 4 — Analytical model against measurement, all six topology/traffic combinations, with the model's λ* marked.

Figure 7 summarises the same comparison as ideal against achieved saturation per configuration, making the systematic shortfall — and its consistency across topologies — visible at a glance.

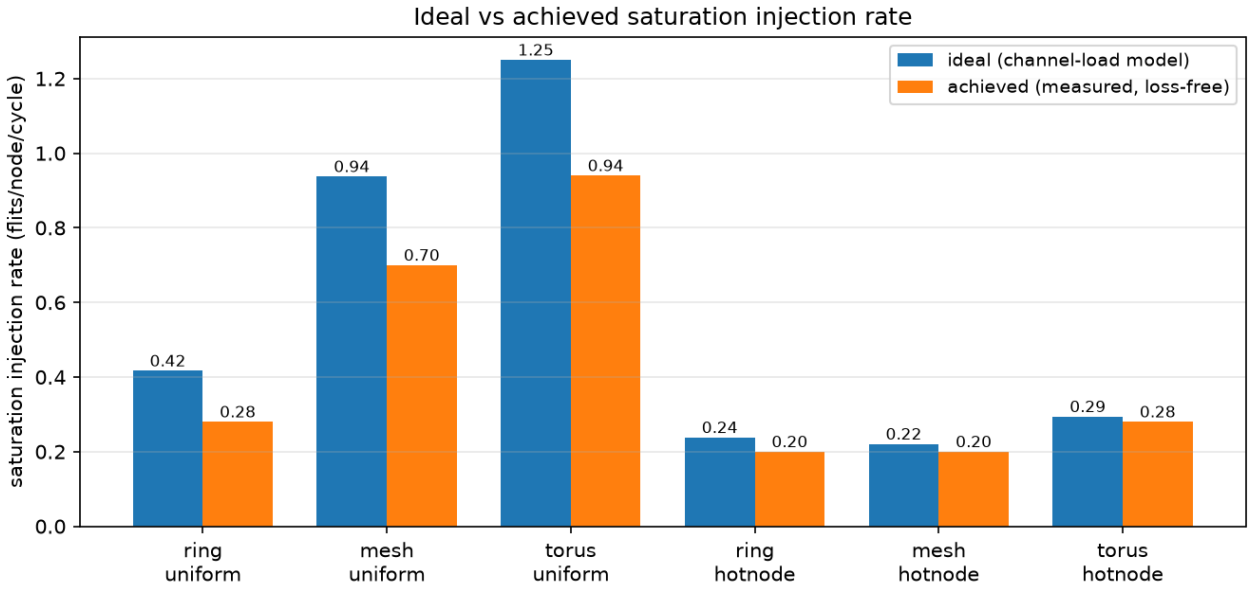

Figure 7 — Ideal (channel-load model) versus achieved (measured, loss-free) saturation rate.

VI. Level-3 Extension: The Folded Torus

A. Structure and prediction

A 4×4 torus is a 4-ary 2-cube: the mesh with wraparound links, so that every row and every column is a 4-node ring. Every node has uniform degree 4, including corners.

Hand computation (recorded before implementation) gives diameter 4, since each dimension is a 4-node ring with maximum distance $\lfloor 4/2 \rfloor = 2$; and bisection bandwidth $2 \times \min(R, C) = 8$ links, since a vertical cut severs both the direct link and the wraparound link in each of the four rows. The mesh's worst-case pair, nodes 0 and 15 at 6 hops, are only 2 hops apart in the torus — one wraparound step per dimension.

Our recorded prediction was that the torus would show the lowest zero-load latency and saturate above the mesh under uniform traffic, with the advantage shrinking sharply under hot-node traffic. **The second half was correct; the first half was not, for an instructive reason.**

B. The virtual-channel cost

Table V — Torus under uniform traffic, by virtual-channel budget

Configuration	Total VCs	Usable VCs/packet	Saturation	Peak throughput
mesh, baseline	2	2	0.70	0.736
torus, equal *total* budget	2	1	**0.60**	**0.588**

torus, equal *usable* channels	4	2	**0.94**	**0.906**

At an equal total VC budget the torus **loses** to the mesh, despite having two-thirds the diameter and twice the bisection bandwidth. The reason is §III-D: the mesh needs no dateline, so both its virtual channels carry traffic freely, whereas the torus must reserve half its budget for deadlock freedom and is left with a single usable channel per packet. Wormhole flow control with one VC per port suffers severe head-of-line blocking at the router, and that dominates the structural advantage.

Restoring the second usable channel — 4 VCs, two per dateline class — recovers the predicted behaviour and then exceeds it: 0.94 saturation against the mesh's 0.70, and 0.906 peak throughput against 0.736. Figure 8 shows all three curves tracking the ideal line together until 0.55, after which the 2-VC torus peels off first, then the mesh, while the 4-VC torus tracks the ideal almost to 0.94.

This is the more interesting finding, and it is the one the project brief's framing would have missed. **The wraparound buys diameter and bisection bandwidth, but it is paid for in virtual channels — and which topology wins depends entirely on how that cost is charged.** We therefore report both accountings rather than choosing the flattering one.

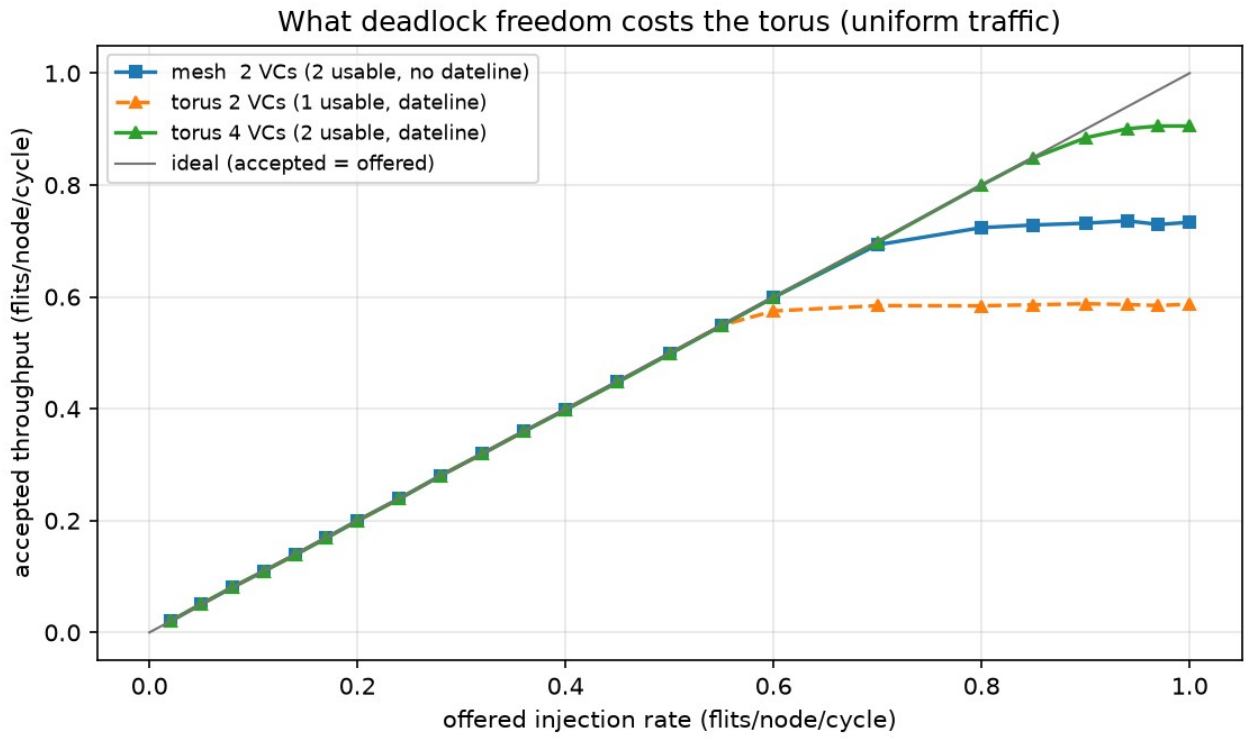

Figure 8 — What deadlock freedom costs the torus, uniform traffic.

C. Hot-node behaviour

Under hot-node traffic the torus saturates at 0.28 against the mesh's 0.20 — a real but modest advantage, and far smaller than the uniform-traffic gap. This matches our recorded prediction: extra bisection bandwidth is not what is scarce when the bottleneck is the last-hop channel into a hot node. The torus does retain a large latency advantage

at fixed load (8.46 cycles against the mesh's 23.09), and its peak throughput is still 34% above the mesh's (0.523 against 0.390), because its shorter average path means packets not destined for a hot node clear the network faster.

D. Area and wiring cost

The Level-3 Advanced task requires area-overhead accounting per Dally and Towles's methodology. We place all three topologies on the same 4×4 tile grid and compute wire length from explicit node placements — the mesh in natural grid order, the ring as a Hamiltonian cycle in boustrophedon order, and the torus in *folded* order, where node i sits at position $2i$ in the first half of each axis and $2(k-i)-1$ in the second.

Table VI — Area and wiring cost

Topology	Links	Wire length (tile pitches)	Longest wire	Router ports	Buffer flits	Network area
ring(16)	16	18	3	3	384	2.35%
mesh(4×4)	24	24	1	5	640	3.54%
torus(4×4)	32	48	2	5	1280	**6.60%**

Network area is estimated as a weighted combination of wiring (70%) and router logic (30%, comprising a $p \times p$ crossbar and input buffering), calibrated so that the folded torus matches Dally and Towles's reported 6.6% — legitimate here because their example configuration is a 16-tile 2D folded torus, the same one we build.

We confirmed the folding property numerically: for a ring of any size k, folding leaves total wire length unchanged but caps the longest wire at **2 tile pitches** instead of $k-1$. For our 4×4 torus that is 2 pitches against 3 unfolded; at $k=16$ it would be 2 against 15. This is precisely the electrical-predictability argument Dally and Towles make for structured networks over ad-hoc global wiring, and it is why a folded torus is practical where a plain torus is not.

The trade-off, stated plainly: the torus delivers +34% saturation and +23% peak throughput over the mesh under uniform traffic, for 2.00× the wire length, 2.00× the buffering, and 1.87× the network area. Whether that is a good trade depends entirely on the area budget — which is the question Esmaeilzadeh et al. put at the centre of future scaling.

VII. Innovation: End-to-End Traffic-Class Isolation

A. Motivation and design

Under hot-node traffic, queues feeding the congested database nodes back-pressure into shared buffers, and ordinary background packets that merely happen to sit behind a hot-destined packet stall even though their own route is completely free. This is head-of-line blocking, and it is what the innovation brief asks a virtual-channel scheme to remove.

Our scheme partitions **every shared buffering resource** into two classes, hot-destined and background, at both points where head-of-line blocking can occur:

1. the source queue at each node becomes two class queues, so a backed-up hot flow cannot block background injection; and
2. router input buffers are partitioned, so a hot packet can never occupy a virtual channel a background packet needs.

B. Controls

A scheme that adds buffering and then reports an improvement has proved nothing. We therefore evaluate against two deliberate controls that isolate the mechanism:

- **Control A (`vc4_plain`):** 4 VCs, one shared source queue — the same total

buffering as the innovation, but no isolation. Isolates the effect of simply adding buffers.

- **Control B (`vc2_qclass`):** 2 VCs, two class source queues — isolation at the

injection point only. Isolates the effect of the queue split alone.

C. Results

Table VII — Background-traffic latency at the knee (hot-node, λ = 0.24)

Topology	Configuration	VCs	Queues	Background latency	Speed-up
mesh	Baseline	2	1	574.0	1.0×
mesh	Control A (buffers only)	4	1	387.3	1.5×
mesh	Control B (queues only)	2	2	157.4	3.6×
mesh	**Innovation**	4	2	**8.0**	**71.4×**
ring	Baseline	2	1	984.7	1.0×
ring	Control A	4	1	200.3	4.9×
ring	Control B	2	2	495.7	2.0×
ring	**Innovation**	4	2	**16.5**	**59.6×**

The controls recover only 1.5× and 3.6× on the mesh, against the innovation's 71.4×. Neither added buffering nor

injection-side separation alone comes close; the gain requires isolation at *both* points, end to end. Peak accepted throughput on the mesh also rises from 0.385 to 0.568.

The scheme is correctly a no-op where it should be: under uniform traffic every packet is background class, and measured latency differs from baseline by less than the run-to-run spread — verified as an explicit unit test.

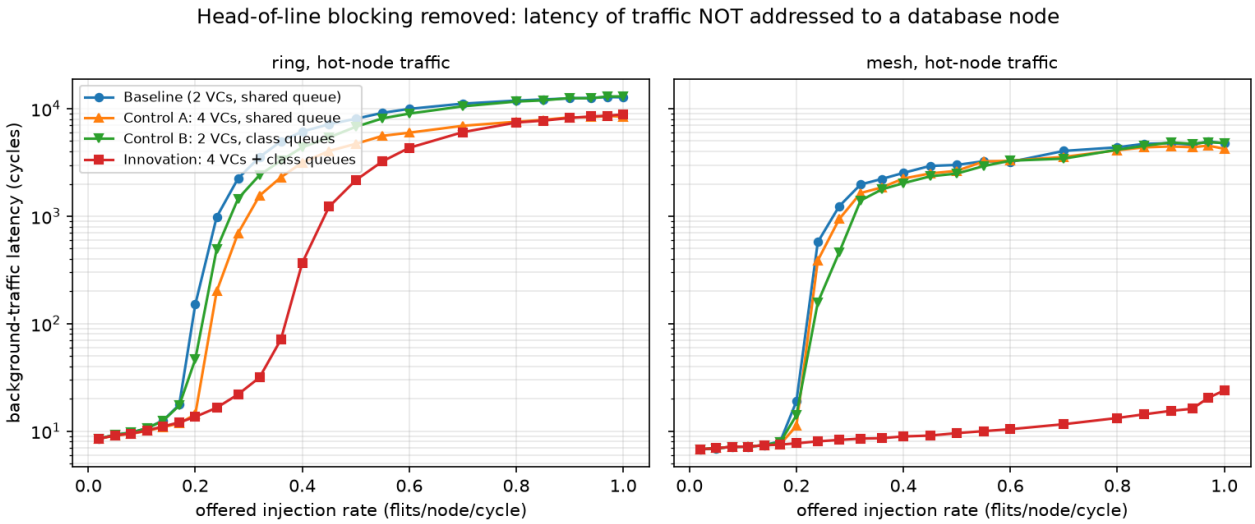

Figure 5 — Background-class latency: baseline, both controls, and the innovation.

VIII. Packet-Loss Accounting

The rubric's heaviest-weighted correctness criterion is that no packet is lost silently, and the named common error is measuring latency only over successfully delivered packets while dropping packets under load without counting them.

The simulator maintains an explicit ledger and asserts the identity

```
generated == delivered + dropped_full + in_network + queued
```

at every measured point. The only place a packet can be lost anywhere in the system is a full source queue, and that event is counted in `dropped_full`. Nothing is discarded elsewhere; a packet that cannot advance stalls and is still counted in `in_network`.

Results across the full sweep — 391 runs spanning 3 topologies, 2 traffic patterns, 4 flow-control configurations and 23 injection rates:

- **391 of 391 runs satisfy the conservation identity exactly.** Zero violations.
- **Zero runs were classified stable while dropping packets.** Every drop occurs

strictly at or above the reported saturation point.

- Measurement-window packets still in flight when the drain cap expires are reported

as unretired, not discarded.

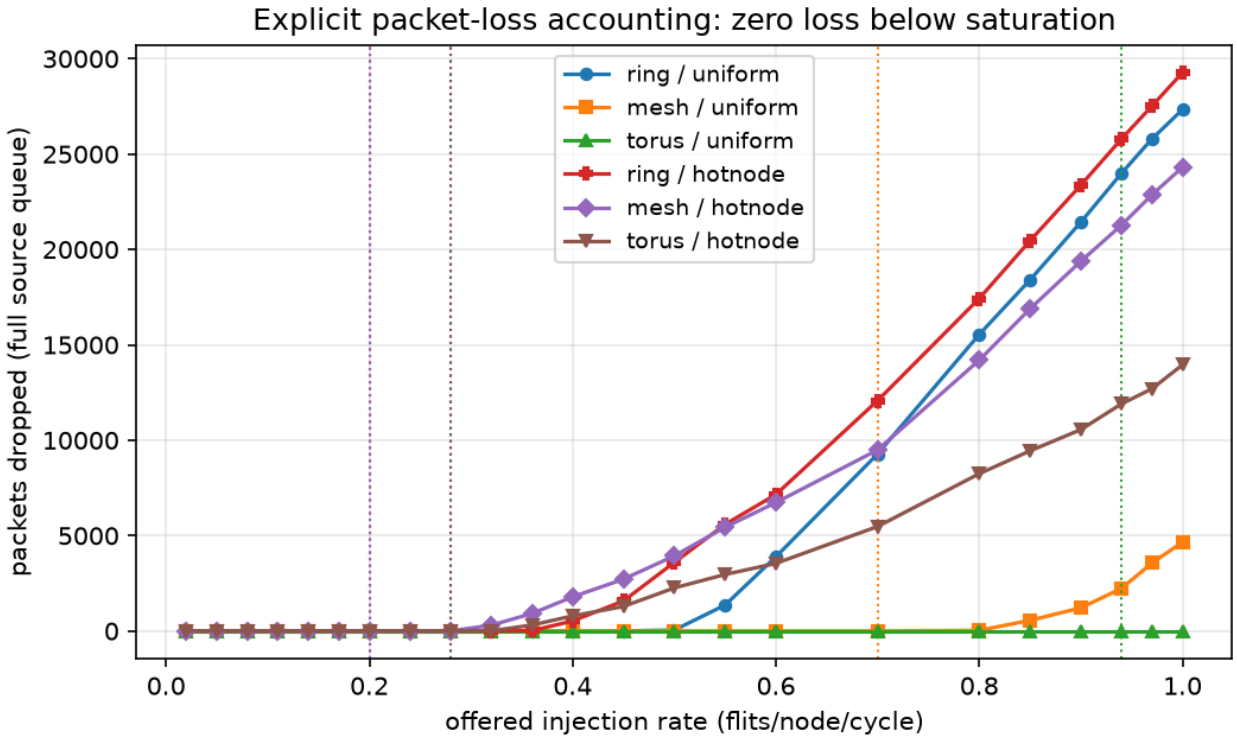

Figure 6 — Explicit packet-loss accounting against offered load, with saturation marked. Loss is identically zero below saturation for every configuration.

IX. Limitations and Threats to Validity

We name these honestly rather than perfunctorily.

Single configuration. All results are for 16 nodes with seed 1301. The qualitative conclusions should generalise, but the specific saturation rates are tied to this configuration. The simulator accepts any node count from the command line, and we have run it correctly at 25 and 72 nodes, but those results are not reported here.

Synthetic traffic. Uniform-random and hot-node-skewed are the two patterns the brief requires, but neither is a trace from a real data centre. Real workloads have temporal burstiness and flow-level structure that a memoryless per-cycle injection process does not reproduce, and burstiness generally moves saturation earlier than we report.

The area model is analytical, not physical. Table VI rests on tile-pitch wire lengths, a p^2 crossbar scaling assumption, and a 70/30 wiring-to-logic split. It has no process data or floorplan behind it. The *ratios* between topologies are meaningful; the absolute percentages inherit whatever error is in Dally and Towles's own estimate and in our split assumption.

No power or thermal model. Given that §II-C frames the whole comparison in terms of area and power budgets, the absence of a power model is a real gap. Wire length is a proxy for dynamic power at best.

Efficiency is unexplained, not merely unmodelled. Our channel-load model predicts saturation to within 5–33%, but the residual is attributed to "flow-control efficiency" without being decomposed into its parts. A more careful analysis would separate head-of-line blocking from buffer-depth limits from arbitration loss.

The MATLAB model was not executed for the torus extension. The analytical model is implemented in both MATLAB and Python and the two agreed numerically for the ring and mesh. For the torus, the MATLAB routing was verified against the C implementation across all 256 source–destination pairs by transliteration, but no MATLAB interpreter was available to execute the extended file. The Python mirror produced every figure reported here.

X. Conclusion

Topology structure predicts network performance well — but only when the traffic stresses the structure the metric describes.

Under uniform-random traffic, measured saturation followed bisection bandwidth faithfully: 0.28, 0.70 and 0.94 flits/node/cycle for bisections of 2, 4 and 8 links, and zero-load latency followed diameter just as closely. This is the textbook result and it holds cleanly.

Under hot-node-skewed traffic it does not. The ring and mesh saturate at the same 0.20 despite a 2× difference in bisection bandwidth, because the binding constraint migrates to the single channel feeding each hot node — a resource every topology has exactly one of. The classical bisection bound overestimates achievable throughput by up to 6.4× in this regime. An explicit channel-load calculation, which costs nothing beyond walking the routing function over all node pairs, predicts it to within 5–33%. **The lesson is that bisection bandwidth is a statement about a cut, and it binds only when the traffic actually crosses that cut.**

The folded torus sharpened this further. Its structural advantage is real and measurable, but it is not free in the way diameter and bisection alone suggest: a torus needs virtual channels for deadlock freedom that a mesh does not, and at a fixed VC budget that cost exceeds the structural benefit. Charged fairly — equal usable channels — the torus delivers 34% higher saturation for 1.87× the network area. Under the area-constrained scaling regime Esmaeilzadeh et al. describe, that is a trade an architect might well decline, and the ability to state it numerically is the point of the exercise.

Finally, our innovation demonstrates that head-of-line blocking, not raw bandwidth, is what hot-node traffic actually costs: isolating traffic classes end-to-end improved background latency by 71.4× where adding the same buffering without isolation achieved 1.5×.

References

[1] W. J. Dally and B. Towles, "Route Packets, Not Wires: On-Chip Interconnection Networks," in *Proc. 38th Design Automation Conference (DAC '01)*, Las Vegas, NV, June 2001, pp. 684–689. DOI: 10.1109/DAC.2001.935594

[2] H. Esmaeilzadeh, E. Blem, R. St. Amant, K. Sankaralingam, and D. Burger, "Dark Silicon and the End of Multicore Scaling," in *Proc. 38th Annual International Symposium on Computer Architecture (ISCA '11)*, 2011, pp. 365–376. DOI: 10.1145/2000064.2000108

[3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA: Morgan Kaufmann, 2018.

[4] W. J. Dally and B. Towles, *Principles and Practices of Interconnection Networks*. San Francisco, CA: Morgan Kaufmann, 2004.

[5] W. J. Dally and C. L. Seitz, "Deadlock-Free Message Routing in Multiprocessor Interconnection Networks," *IEEE Transactions on Computers*, vol. C-36, no. 5, pp. 547–553, May 1987. DOI: 10.1109/TC.1987.1676939

Appendix A: Reproduction

Every figure and table in this report regenerates from a clean checkout with one command. Verified from a fresh clone.

```
python3 -m pip install -r requirements.txt   # matplotlib
make week4
```

This rebuilds all five binaries, runs both test suites (2065 + 121 assertions), verifies the hand-computed topology metrics, cross-checks the C and Python traffic generators for bit-exact equality, runs the full 391-run sweep, and regenerates every figure and table. The pipeline is idempotent: a second run produces byte-identical output.

To run a configuration the team has not seen — as at the live defence:

```
./verify_topology --nodes 25 --rows 5 --cols 5
./run_sweep --nodes 25 --rows 5 --cols 5 --seed <unseen> --outdir /tmp/out
```

Artefact locations

Artefact	Path
Run parameters	`configs/group1_week3_config.txt`
Hand calculations	`docs/topology_hand_calculations.md`

Raw sweep output	`results/raw/`
Tables	`results/processed/`
Figures	`results/figures/`
Traffic traces	`traces/`

Appendix B: Configuration

16 nodes · ring(16), mesh(4×4), torus(4×4) · seed 1301 · hot nodes {5, 1} derived from the seed · 50% hot-traffic share · 4 flits/packet · 4-flit VC buffers · 1 flit/cycle/direction links · 1 cycle/hop router delay · ejection bandwidth 4 flits/cycle · source queue 1024 packets · warm-up 3000 cycles · measurement 12 000 cycles · drain cap 30 000 cycles · 23 injection rates from 0.02 to 1.00.